new point or fail, otherwise we have a wide range of solutions, from using a counter on nodes to keep track of how many times a point was added, up to consistently add duplicates to either branch of a node (for instance, always in the left sub-branch: although, as we have seen in appendix C, this leads to slightly unbalanced trees on average).

If, instead, the point was not on the tree, search will fail on the node that should have the new point as its child, and the second step will consist in creating a new node for the point and adding it in the correct branch of its parent.

Listing 9.4 shows an approach that doesn't reuse the search method. This approach, while not DRY[4], allows us to simplify both methods: to be reusable in `insert`, `search` should also return the parent of the node found (and even more when it's not found), which is not of any particular use for search itself, that would thus become unnecessary complicated. Moreover, this way we can write insert in a more elegant way, using a pattern that naturally support immutability[5].

Listing 9.4 The insert method

```
function insert(node, newPoint,
level=0)                                #1
  if node == null
then                                                  #2
    return new KdNode(newPoint, null, null, level)
  elsif node.point == newPoint
then                                     #3
    return node
  elsif compare(newPoint, node) < 0
then                                  #4
```